Verdict-Preserving Configuration Fidelity and Selective Synchronization for Security Digital Twins

Author Name(s)
Department, Organization
City, Country
email@organization.org

***Abstract*—Security digital twins check whether a production service is exposed by running its configuration in isolation. In practice a twin platform rarely receives the whole configuration: copying everything leaks customer data and forces a costly re-emulation on every edit, while copying too little, or filling gaps with platform defaults, yields confidently wrong verdicts. We ask which directives a security verdict actually depends on. We define *fidelity cores* (minimal directive sets that reproduce a verdict), *pivots* (directives whose removal alone inverts it) and *guard sets* (keys whose edits require re-emulation), and prove when guard-based synchronization is exact. Measured on real Redis, PostgreSQL, Nginx and OpenSSH servers and 16 pinned builds, over 240 seeded configurations with 14,400 edits, 1,092 public files and 426 real commits, no core exceeded two directives, yet a random 90% copy was wrong 6.6% of the time because 39% of pivots lie outside the core. Guard sets learned from synthetic data reproduced all 450 tested public verdicts while exporting 24% of Redis and 12% of OpenSSH directives, cut real re-emulations from 426 to 137 without missing a verdict change, and matched documentation-derived dependency selection at lower cost. Version numbers hide two hazards: Redis 7.0.15 images and Ubuntu packages default protected mode oppositely, and a platform default profile misjudged 22% of public sshd files at full synchronization. Declarative specifications extend the analysis to new services without code.**

***Index Terms*—digital twin, configuration fidelity, security validation, delta debugging, configuration drift, misconfiguration**

I. Introduction

Most of a configuration file is housekeeping: buffer sizes, log formats, keep-alive timers. Whether a service is exposed usually comes down to a handful of lines, sometimes just one. A twin that copies the whole file is faithful, but expensive to build, to keep in sync and to trust with customer data. A twin that copies too little gives answers that are confidently wrong. Our earlier configuration-synchronized twin platform [1] showed the second failure: twins rendered from default settings reached an F1 score of only 0.40, while synchronized twins were correct on every conclusive run. That told us *that* configuration matters, but not *how much* of it a verdict needs, or *which* part.

The answer matters in practice. Every exported directive widens what a shared control plane holds about a customer; every change triggers a costly re-emulation if the platform watches content hashes; and an operator who receives a finding wants to know which lines cause it and which lines are holding the line. Inspection is a poor guide. In the Nginx configuration of Fig. 1, a server-wide `deny all` and HTTP basic authentication look protective, yet `/admin/` is open, because `satisfy any` lets a matching `allow 10.77.0.1` stand in for authentication. Those two lines decide the verdict, the protective-looking ones can each be removed without effect, and a twin that drops either of the two gets the answer wrong.

Aim. This work aims to make security twins *minimal yet correct*: to determine which part of a configuration a security verdict depends on, and to use that knowledge to decide what a twin must copy and when it must be re-run. We pursue three objectives. *Fidelity*: how accuracy degrades under partial copying (**RQ1**), how large, unique and complete minimal verdict-preserving subsets are (**RQ2**), and how versions and builds change verdicts (**RQ5**). *Synchronization*: whether twins can be built from a learned key set (**RQ3**) and when to re-emulate under drift, compared with dependency-aware selection (**RQ4**). *Generalization*: whether findings hold on public configurations (**RQ6**), whether careful static analysis could replace the twin (**RQ7**), and whether new services can be added without code. Our contributions are: a formal model of verdict-relative fidelity with conditions under which guard-based synchronization is exact (Section III); a harness that obtains every verdict from real servers or pinned builds through wire probes (Section IV); three corpora (240 seeded configurations with 14,400 edits, 1,092 public files, 216 public histories with 426 commits); declarative service specifications that extend everything to new services without code (Section VI); and findings distilled into design rules (Sections V and VII).

The results pull in two directions. Verdicts are *concentrated*: no core in any corpus exceeded two directives. They are also *brittle*: a random 90% copy was wrong one time in fifteen. The two fit together once one sees that the lines keeping a service safe are often not the lines that make it unsafe, so a minimal explanation of a verdict says little about what will break it.

II. Background and Related Work

Digital twins and fidelity. The digital twin grew out of product lifecycle management [2] and manufacturing [3] and was adopted for cyber-physical security [4], [5], including

```
1  server_tokens off;
2  auth_basic "Restricted";
3  autoindex off;
4  add_header X-Content-Type-Options
5  satisfy any;
6  allow 10.77.0.1;
7  client_max_body_size 8m;
8  deny all;
9  keepalive_timeout 15;
```

φ_1: is `/admin/` reachable?	φ_2: is the version disclosed?
verdict **yes**; core $= \emptyset$ (Nginx is open by default); pivots: lines 5 and 6.	verdict **no**; core $= \{1\}$; pivot: line 1.

Fig. 1. A configuration from the synthetic corpus, with block structure flattened for space. Shaded lines are *pivots* for φ_1: removing either one causes the twin to report "closed". The ostensibly protective lines 2 and 8 can each be removed without affecting the verdict.

simulation and replication of industrial control systems [6], as both defensive tool and attack surface [7], and across security operations [8]. A National Academies report argues that a twin should be built at the fidelity that makes it *fit for purpose* [9]; since a security twin's purpose is a verdict, we measure fidelity per verdict. CrystalNet emulates production networks with real firmware and notes that complete fidelity would mean replicating production [10]; cyber ranges face similar trade-offs [11].

Configuration analysis, checking and testing. Batfish [12], Minesweeper [13] and Config2Spec [14] reason about network configurations without running devices, and attack-graph tools encode vulnerability interactions as logic over configuration facts [15], [16]; all depend on a correct semantic model, which **RQ7** tests. Misconfiguration is a leading cause of failures [17] and most knobs are rarely set [18]. PeerPressure compares a machine with its peers [19], EnCore learns correlations among settings and the environment [20], PCheck emulates late uses of configuration values at startup [21], ConfErr injects human-like errors [22] and Rabkin and Katz precompute diagnoses [23]. Configuration-aware regression testing selects the tests a change can affect: Ctest runs the tests that exercise changed parameters [24], uRTS unifies selection for code and configuration changes [25], Ctest4J makes such tests practical in Java builds [26], and ConfFLARE uses data-flow analysis to restrict performance testing to the configurations a change can touch [27]. These dependency-based selectors are the closest relatives of our drift guard, and **RQ4** compares against one; they target functional or performance failures rather than security verdicts, and need a dependency map our guards learn. Recent security-configuration work studies faults in infrastructure-as-code ecosystems [28] and repairs IaC misconfigurations with SMT-based search [29], but reasons over scripts rather than executing the configured service. Options often matter only in combination [30], which a core larger than one captures; practice surveys [31], infrastructure-as-code smells [32] and industrial configuration management [33] complete the picture.

Minimization. Delta debugging reduces an input to a 1-minimal failing subset [34] with a quadratic worst case [35]; later work exploits structure [36] and speeds it up [37]. Program slicing keeps only statements relevant to a value [38]. A fidelity core is a verdict-preserving slice of a configuration, computed by delta debugging. No prior work we know asks which part of a configuration a security verdict needs, or uses that answer to decide what to copy and when to re-run.

III. Formal Model

A *configuration* $C = \langle d_1, \ldots, d_n \rangle$ is an ordered sequence of directives $d = (\sigma, k, x)$ with section σ, key k and value x. Order is semantic: pg_hba uses the first matching rule, sshd the first value for a key, Redis and `postgresql.conf` the last. A *base profile* B fills section–key slots a twin was not given: the *vendor* profile V is empty (compiled-in defaults); the *platform* profile P reproduces our earlier default twin [1] (Redis protected mode on, SSH password login off, PostgreSQL listening everywhere with a SCRAM catch-all, Nginx version tokens on). For $S \subseteq C$, the twin $T_B(S)$ renders S followed by the entries of B for unassigned slots. A *property* φ is a yes/no question answered over the wire; $v_\varphi(\cdot)$ is the verdict of a real server and $v^\star = v_\varphi(T_\mathsf{V}(C))$ the source verdict. A server that fails to start yields *error*, never equal to $v^\star$.

Definition 1. *The* fidelity curve $\varepsilon_B(p)$ *is the probability that* $v_\varphi(T_B(S)) \neq v^\star$ *for uniformly random* $S \subseteq C$ *with* $|S| = \mathrm{round}(pn)$. *A* fidelity core *is a* $K \subseteq C$ *with* $v_\varphi(T_B(K)) = v^\star$ *and* $v_\varphi(T_B(K \setminus \{d\})) \neq v^\star$ *for all* $d \in K$. *A* pivot *is a* $d \in C$ *with* $v_\varphi(T_\mathsf{V}(C \setminus \{d\})) \neq v^\star$.

A core captures *sufficiency*, a pivot *necessity*. They differ in three ways that matter later. Cores are not upward-closed: for Redis with `protected-mode no` followed by `requirepass`, the empty set is a core (both source and empty twin are safe) but {`protected-mode no`} is exposed, so "the core plus more" guarantees nothing. Cores are sufficient but not causal: an sshd `PasswordAuthentication no` that follows an earlier `yes` is ignored by the server, yet alone it forms a valid core. And they differ in cost: ddmin finds a 1-minimal core in $O(n^2)$ verdict queries (logarithmic when one directive decides) [35], a minimum core may need exhaustive search, and pivots need exactly n queries. When even $T_B(C)$ disagrees with the source we record a *base conflict*.

A *guard set* G_φ is a set of keys. *Selective synchronization* exports only $C|_G$, the directives with keys in G; *key-guarded re-emulation* re-runs the twin only after edits that touch a key in G.

Definition 2. φ *is* G-local *over a family if* $C|_G = C'|_G$ *implies* $v_\varphi(C) = v_\varphi(C')$. *A base* B *is* G-silent *if it has no entry with a key in* G.

Proposition 1. *If* φ *is* G*-local, then* $T_\mathsf{V}(C|_G)$ *yields the source verdict, and key-guarded re-emulation never reports a stale verdict. If in addition* B *is* G*-silent,* $T_B(C|_G)$ *yields the source verdict.*

Algorithm 1 Fidelity core and pivots

Input: configuration C, base B, property φ, server oracle v_φ
Output: core K (or CONFLICT), pivot set Π

$v^\star \leftarrow v_\varphi(T_\vee(C))$
if $v_\varphi(T_B(C)) \neq v^\star$ **then return** CONFLICT
end if
$ok(S) \leftarrow [\, v_\varphi(T_B(S)) = v^\star \,]$ ▷ memoized
if $ok(\emptyset)$ **then** $K \leftarrow \emptyset$
else $K \leftarrow \text{DDMIN}(ok, C)$
end if
$\Pi \leftarrow \{\, d \in C : v_\varphi(T_\vee(C \setminus \{d\})) \neq v^\star \,\}$
return K, Π

TABLE I
SECURITY POLICIES AND VERDICT ORIENTATION

Property	Policy (violated when the verdict is *positive*)	Wire probe
Redis	no unauthenticated remote client may run admin commands	`CONFIG GET` via `redis-cli`
PostgreSQL	no remote role may run SQL without a password	`psql -w`, role `synthetic`
Nginx leak	the server must not disclose its version	`Server` header of `GET /`
Nginx admin	`/admin/` must require authorization	status of `GET /admin/`
OpenSSH	an ordinary account must not log in by password	`sshpass+ssh`

Positive (*exposed*) = probe succeeds; negative (*safe*) = recognized refusal (e.g. `NOAUTH`, "no pg_hba.conf entry", 401/403, "Permission denied"); anything else is retried once, then inconclusive. Exposure is the positive class for precision and false-alarm counts.

Proof. $T_\vee(C|_G)$ is $C|_G$, whose restriction to G is itself. An edit touching no key in G leaves $C|_G$ invariant, including the relative order of its directives, so the verdict is unchanged; every other edit triggers re-emulation. If B is G-silent, $T_B(C|_G)$ adds only non-G entries, so its restriction to G is still $C|_G$ and locality applies. □

Any base becomes G-silent if the agent reports *absence*: for each guard key unset in the source, the twin suppresses the base value and falls back to the server default. Algorithm 1 computes cores and pivots; we learn G_φ as the union of their keys over training configurations, optionally with an expert key list as a floor.

IV. Experimental Methodology

Harness and setup. Each evaluation renders the configuration, starts a fresh server, waits for readiness (a log marker, socket or port), probes it from a non-loopback client (10.77.0.1) with the stock client, and stops it. Ports, data directories, host keys and document trees are fixed per-slot scaffolding, never fidelity variables; four worker slots with disjoint ports run in parallel. Table I gives each property's policy and verdict orientation; no outcome stayed inconclusive after retry. Start-up timeouts are 8–40 s and probe timeouts 4–25 s per service; the synthetic users and the single synthetic password exist only in the twin. Seeds are fixed (corpus 2026, splits by index parity, drift seeds per trace), and a verdict cache keyed by the canonical twin removes repeated runs. Experiments ran on Ubuntu 24.04 (Linux 6.18, Python 3.13, Docker 29.9.2). Reference servers were the Ubuntu 24.04 builds of Redis 7.0.15, PostgreSQL 16.15, Nginx 1.24.0 and OpenSSH 9.6p1 on a 4-vCPU, 16 GB container. A native evaluation took a median of 26–161 ms, about 60,000 in total; re-running 1,000 cached twins on fresh processes reproduced all 1,273 verdicts. For **RQ5**, 16 pinned container builds run in the host network namespace: Redis 6.0, 6.2, 7.0, 7.2 and 7.4; PostgreSQL 12, 14, 16 and 17; Nginx 1.20, 1.24 and 1.27; and OpenSSH 8.8p1, 9.3p2, 9.7p1 and 10.0p2. Fidelity curves use $p \in \{0, 0.1, \ldots, 1\}$ with eight random subsets per level.

Synthetic corpus and drift. No public corpus pairs configurations with wire-verified verdicts, so we generated one from a seeded catalog of legal sections, values and a *security*/*operational* label. Each configuration combines 3–10 operational directives with a *posture*, a recipe modeled on a recurring pattern (protected mode off for a container network, a pg_hba trust rule for a CI subnet, a `deny all` reopened in one location, an appended sshd override that never takes effect). We generated 90 candidates per family (360). A candidate becomes a source only if the real server starts and answers conclusively; 357 passed, the three failures being sshd files that demanded a disabled `AuthenticationMethods` method. The first 60 admitted per family in seed order form the corpus (240, 5–16 directives), split by index parity into 120 training and 120 held-out configurations. Labels are used only for the expert guard and the drift mix. Each held-out configuration receives three 40-edit traces with security-edit share $q \in \{0.1, 0.3, 0.5\}$; an edit modifies a directive (probability 1/2) or inserts, deletes or moves one (1/6 each), and server-rejected edits are rolled back (32 times).

Public corpus. GitHub code search yielded 2,363 `redis.conf`, `sshd_config` and `pg_hba.conf` files. A fixed pipeline rejected templates and manual pages (468), removed environment-bound directives (ports, paths, includes, modules, TLS material, PAM, replication peers), remapped LAN addresses to the twin, replaced every secret with a synthetic password, and studied pg_hba with the server listening everywhere; deduplication left 1,092 configurations (420 Redis, 339 PostgreSQL, 333 OpenSSH). Blob-less clones of 1,500 sampled files gave 216 histories with 698 versions and 426 commits. Nginx was excluded because real files depend on site-specific includes.

Analyzers and metrics. For **RQ7** we wrote two analyzers from documentation before any comparison: a *naive* one mirroring the platform's original baseline (last value of a few keys) and a *careful* one encoding last-/first-wins keys, pg_hba first match, Nginx access-rule inheritance, sshd `Match` precedence and Redis ACL tokens (extended once, before evaluation, to the full pg_hba column syntax). Error rates are averaged per configuration; 95% intervals use a cluster bootstrap with 2,000 resamples. For drift we report re-emulations per 100 edits, stale steps and *late flips* (verdict changes not caught at the causing edit).

TABLE II
CORES (SUFFICIENCY) VERSUS PIVOTS (NECESSITY), BASE V

Corpus	φ	Configs	$\overline{n}$	$\overline{\lvert K\rvert}$	max $\lvert K\rvert$	Pivots	in K
Synthetic	Redis	60	9.0	0.33	1	43	20
	PostgreSQL	60	10.8	0.47	2	33	26
	Nginx leak	60	10.4	0.50	1	27	27
	Nginx admin	60	10.4	0.48	1	62	22
	SSH	60	10.2	0.72	1	29	23
Public	Redis	150	21.4	0.17	1	36	26
	PostgreSQL	150	6.7	0.21	2	33	32
	SSH	150	13.1	0.69	1	60	58

$\overline{n}$: mean directives. Pivots: single removals that invert the verdict; "in K": of those, in the ddmin core. Public: seeded sample of admitted files.

TABLE III
SELECTIVE SYNCHRONIZATION WITH LEARNED AND EXPERT GUARD SETS

	Synthetic held-out			Public, transfer (V)		
Guard set	Exp.	Acc. V	Acc. P	Redis	SSH	PG
G_{core}	20.5%	91.4%	86.4%	90.7%	100%	100%
G_{pivot}	23.8%	98.4%	93.7%	–	–	–
$G_{\text{core}} \cup G_{\text{pivot}}$	24.1%	99.3%	94.3%	100%	100%	100%
G_{expert}	34.8%	100%	94.9%	100%	100%	100%
Exported share, public ($G_{\text{core}} \cup G_{\text{pivot}}$)	23.8%		12.3%			100%

Synthetic: 900 held-out verdicts per row. Public: 150 files per family, guard sets learned on synthetic data and applied unchanged.

V. RESULTS

A. *RQ1: Partial fidelity is brittle*

Error falls roughly in a straight line as more directives are copied (Fig. 2); there is no point where copying "most" of a file is good enough. Under V, synthetic error was 45.3% with nothing copied, 28.3% at half (95% CI 25.2–31.5%), 11.6% at 80% and 6.6% (5.5–7.8%) at 90%. SSH starts highest (71.7%) because the vendor default permits password login; PostgreSQL stays near 23% until most rules are present, since a trust rule matters only with `listen_addresses` and every earlier matching rule. Public files show the same shape: pooled error was 32.2% with nothing copied, 14.1% at half and 3.4% at 90%.

Which way the twin errs depends on the base. Under V, false alarms outnumbered false-safe verdicts 3,658 to 1,980, because random cuts drop defensive lines and leave exposures standing. P halves SSH error at low fidelity but never reaches zero: at full synchronization 1.3% of synthetic verdicts were wrong, all SSH sources that relied on the vendor default for `PasswordAuthentication`, and in 88 runs the platform value stopped sshd from starting by contradicting `AuthenticationMethods`. On public sshd files the effect was far larger: 33 of 150 (22%) had no core under P. A base profile is not a neutral filler; it is configuration the source never had.

B. *RQ2: Cores are tiny and usually unique but miss pivots*

Across 300 synthetic configuration–property pairs under V, the largest core held two directives, the median zero and the mean 0.50 (5.1% of a file); size-two cores occurred only for PostgreSQL exposures (`listen_addresses` plus the first trust rule). Exhaustive search over all subsets of up to two directives confirmed the ddmin core was a *minimum* core in all 300 pairs under V and 296 under P; it was unique in 266 (89%), and the other 34 had two or three, 20 of them SSH logins blocked for several independent reasons. A core cost 3.2–5.1 server runs on average (at most 22), and 2.6–5.6 on public files of up to 87 directives (at most 24).

Small cores do not mean few lines matter (Table II). Of 194 synthetic pivots, 76 (39%) lay outside the vendor core; for Nginx `/admin/` only 22 of 62 did. Conversely, 79–91% of security-labeled directives were in no core, among them `PermitRootLogin`, `PubkeyAuthentication` and shadowed pg_hba rules: a security label describes a key, a core describes one verdict. Public files, though longer, had equally small cores; in public Redis files 10 of 36 pivots (`requirepass`, ACL `user` lines, `rename-command`) were outside the core, each keeping a server with `protected-mode no` safe.

C. *RQ3: Selective synchronization and transfer*

Learned from cores alone, the Redis guard is just `protected-mode`, and held-out accuracy drops to 91.4% because cores never see the lines that prevent exposure (Table III). Adding pivot keys gives 99.3% at a 24% export share; the six residual errors involve `bind` and `DenyUsers`, never decisive in training, which are exactly the locality failures Proposition 1 predicts for a learned guard. The expert list reached 100% at 35%. Applied unchanged to public files, the union guard reproduced all 450 verdicts while exporting 23.8% of Redis and 12.3% of OpenSSH directives (pg_hba is exported whole). Under P every guard lost about five points: the expert set's 46 errors were 26 SSH false-safe verdicts (absent `PasswordAuthentication` filled with "no"), 16 PostgreSQL false alarms (absent `listen_addresses` filled with "*") and 4 SSH start failures, exactly what absence markers remove (Proposition 1).

D. *RQ4: Drift and re-emulation*

Re-emulating on every hash change is never stale, but at $q = 0.3$ the verdict flipped on only 4.8 of every 100 edits (Table IV). A fixed schedule did badly: every fourth edit cost 25 runs per 100 edits, left 6.35% of steps stale and caught only 93 of 289 flips at once. The learned union guard used 19.9 runs per 100 edits, five times fewer than hash-triggered, at 0.28% stale steps; its four late flips were `bind` or `DenyUsers` edits. The expert guard was never stale at 29.7. Guards also specialize: for Nginx version disclosure the learned guard is just `server_tokens` (6.2 runs per 100 edits). Audits and online growth cut staleness to 0.13% and 0.03% at about the expert cost. The ordering held at $q = 0.1$ (union 6.5 runs, 1.05% stale; expert 9.9, none) and $q = 0.5$ (32.2, 0.58%; 49.6, none). Real histories behave like a low-q trace with bigger edits: of 426 commits, 180 touched only comments or

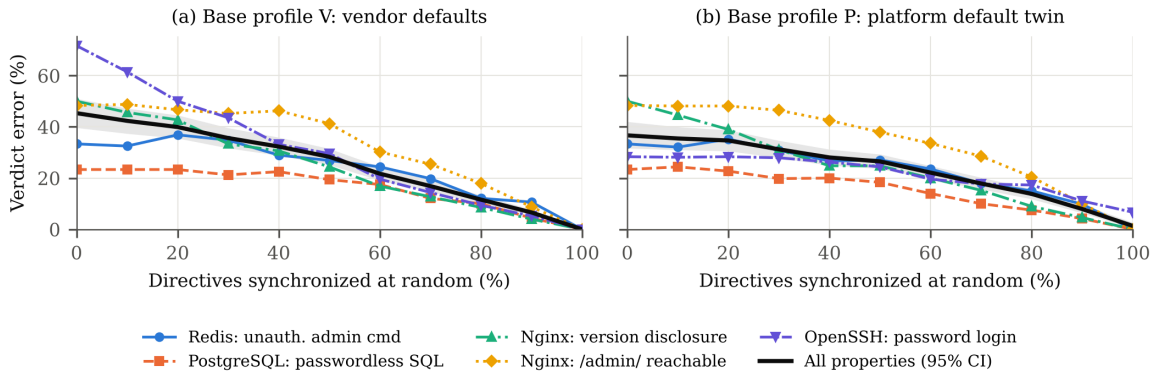


Fig. 2. Fidelity curves on the synthetic corpus: verdict error of twins mirroring a random fraction of directives on (a) vendor defaults and (b) the platform profile. Black: all properties, 95% cluster-bootstrap interval.

TABLE IV
RE-EMULATION POLICIES (SYNTHETIC, $q = 0.3$; PUBLIC HISTORIES)

Policy	Re-emul./100	Stale steps	Late flips
Synthetic: 4,800 edits, 289 true flips			
Hash-triggered (current)	99.8	0.00%	0
Periodic, $k = 2$	50.0	2.38%	143
Periodic, $k = 4$	25.0	6.35%	196
Periodic, $k = 8$	12.5	13.38%	221
Guard G_{core}	15.8	2.00%	24
Guard G_{pivot}	19.5	1.15%	14
Guard $G_{\text{core}} \cup G_{\text{pivot}}$	19.9	0.28%	4
Guard G_{expert}	29.7	0.00%	0
Dependency-aware	21.4	0.00%	0
Union guard + audit every 8	29.8	0.13%	4
Online-grown guard	34.8	0.03%	1
Oracle (lower bound)	4.8	0.00%	0
Public histories: 426 commits, 19 true flips			
Per commit	100	0.00%	0
Per directive-hash change	57.7	0.00%	0
Guard $G_{\text{core}} \cup G_{\text{pivot}}$	32.2	0.00%	0
Guard G_{expert}	37.3	0.00%	0
Dependency-aware	34.3	0.00%	0
Oracle (lower bound)	4.5	0.00%	0

environment-bound lines and 19 flipped a verdict (7 Redis, 7 OpenSSH, 5 PostgreSQL). The union guard needed 137 runs and the expert guard 159, against 426 per commit and 246 per directive hash, and both caught every flip at its commit. Replayed as a change gate, 10 of these commits introduced an exposure (5 Redis, 4 PostgreSQL, 1 OpenSSH), and the gate blocked all 10.

Comparison with dependency-aware selection. Configuration-aware test selection [24], [25] re-runs a check only when an edit touches a parameter the check depends on. We built that baseline from documentation: the keys each property's documented semantics read (the inputs of our careful analyzer, Section V-G). It was never stale, needing 7.1, 21.4 and 35.2 re-emulations per 100 synthetic edits at $q = 0.1, 0.3, 0.5$ and 146 on the real commits, and as a selective-sync set it reproduced all 900 held-out verdicts while exporting 25% of directives. It thus sits between the learned union guard (cheaper, slightly stale) and the expert list (always correct, about 40% costlier), and confirms that most of the savings come from knowing *which keys* a verdict reads. Its limit is how it is obtained: an analyst must derive the map for every service and keep it current, whereas core-and-pivot guards are learned from the servers themselves and were 6–7% cheaper, at the price of four late flips on synthetic drift.

E. *RQ5: Versions and builds*

Replaying 120 configurations per family (60 corpus, 60 drift versions) on every pinned build, PostgreSQL 12–17, Nginx 1.20–1.27 and OpenSSH 8.8–10.0 returned the reference verdict everywhere, with identical core keys; for these properties version is not a fidelity variable. Redis showed three distinct hazards. *Syntax*: 6.0 refused 26 configurations using later syntax (24 ACL `&*` selectors, two `bind * -::*`). *Semantics*: eight configurations flipped from exposed on 6.2 to safe on 7.0, because before 7.0 an explicit `bind` disabled protected mode. *Build defaults*: the official 7.0.15 image disagreed with the Ubuntu 7.0.15 package on 11 configurations, all leaving `protected-mode` unset; every official image (6.0–7.4) defaults it to `no`, the Ubuntu build to `yes`. Redis core keys were identical across builds for 56 of 60 configurations. A twin pinned to "Redis 7.0.15" can thus give opposite answers depending on where its binary came from: matching the version is necessary but not enough.

F. *RQ6: Public configurations*

A source must run before it can be twinned. The reference servers rejected 45 of 420 public Redis files (10.7%, mostly directives of other versions), 43 of 333 sshd files (12.9%: invalid combinations, options newer than OpenSSH 9.6 such as post-quantum key exchange, seven that never became ready, and six requiring keyboard-interactive methods our PAM-less twin lacks) and 10 of 339 pg_hba files (3%). Among admitted files, 72 of 375 Redis configurations (19%) accepted unauthenticated administrative commands, 34 of 329 pg_hba files (10%) admitted an unlisted role without a password and 95 of 290 sshd files (33%) allowed password login, all in the twin setting rather than the unknown deployments. Cores

TABLE V
STATIC ANALYZERS VERSUS REAL-SERVER VERDICTS

	Synthetic, naive		Synthetic, careful		Public
φ	Acc.	FP / FN	Acc.	FP / FN	careful
Redis unauth. admin	88.4%	392 / 4	99.82%	2 / 4	99.2%
PG passwordless SQL	74.5%	868 / 0	100%	0 / 0	100%
Nginx version leak	92.7%	157 / 94	100%	0 / 0	–
Nginx /admin/ open	76.1%	127 / 691	99.77%	8 / 0	–
SSH password login	72.3%	843 / 90	100%	0 / 0	99.0%

Synthetic: 13,606 distinct configurations. Public: 375 Redis, 329 PostgreSQL, 290 OpenSSH files.

TABLE VI
SPEC-ONLY SERVICES (40 CONFIGURATIONS, 400 DRIFT EDITS EACH, V)

Service	Exp. max $\lvert K\rvert$	Piv. (in K)	Expert	H / E / O
Mosquitto	9	2	56 (24)	100%/37%	93.2/29.8/3.5
MongoDB	17	1	24 (17)	100%/30%	99.5/27.8/6.0
MariaDB	11	1	29 (11)	100%/26%	99.0/27.5/8.5

Expert: held-out accuracy / exported share. H/E/O: re-emulations per 100 edits ($q = 0.3$) for hash, expert guard, oracle; the expert guard left 0.25% of Mosquitto steps stale.

stayed at most two directives in every family, and the curves and guard transfer reported above held.

G. *RQ7: Static analysis versus execution*

If only one or two lines matter, why not just read them? The naive analyzer was right only 72–93% of the time (Table V): it ignored sshd's first-value rule, pg_hba ordering and scoping, and Nginx's rule that any inner access directive replaces the parent's list. The careful analyzer agreed on 17,018 of 17,032 synthetic verdicts, and each of its 14 errors traced to documented behavior that is easy to misread. Redis ACL lines *rebuild* the user, so `user default on nopass ~* &* +@read +@write` cannot run `CONFIG`; an ACL `nopass` line beats a later `requirepass` regardless of order (four missed exposures); and under Nginx `satisfy any` the implicit allow does not satisfy access, so the server answers 401. On public files it reached 99.0–100%, but its six errors were new: three Redis files defined extra ACL users the analyzer applied to the default user, two sshd files used a wildcard in `AllowGroups` or space-separated `AuthenticationMethods` alternatives, and one was a probe artifact (`ForceCommand`). A static model must be extended for every bit of syntax it meets in the wild, while a twin uses the server's own parser.

VI. GENERALIZATION: DECLARATIVE SERVICE ADAPTERS

To check that none of this is specific to four hand-written adapters, we replaced them with declarative YAML specifications. A spec declares file templates and line formats (flat key–value, INI with flags, nested YAML via dotted keys, brace blocks, rule files, dynamic `Match` blocks), a parser for real files, the runtime (native binary or pinned image), readiness, preparation steps and wire probes with verdict regexes; cores, pivots, curves, guards and drift policies run unchanged on any spec. Specs for the original four services reproduced all 614 verdicts of the hand-written adapters on the 240 corpus configurations and 240 drift versions.

Adapter effort and limits. Excluding the optional directive catalog, a spec takes 37–57 non-comment YAML lines (Redis 37, OpenSSH 43, MongoDB 44, PostgreSQL 51, MariaDB 52, Mosquitto 52, Nginx 57), against about 54 lines of Python per hand-written adapter; catalogs add 15–18 lines for the new services, and 14–30 environment-bound keys per service had to be declared. Each new spec ran end to end after one round of fixes. Unsupported by design are anything a twin cannot supply (PAM stacks, TLS material, replication and cluster peers, site-specific includes and file paths) and properties not observable by one wire probe; the brace-block parser for real Nginx files is not implemented. The service-specific failures we hit were: mongod reading YAML 1.2 (`off` is a string) where Python's default YAML 1.1 reads a Boolean, readiness lines hidden by quiet log levels in Redis and Mosquitto, keyboard-interactive sshd methods that a PAM-less twin rejects, and MongoDB option constraints broken by normalization (Section V-F).

Mosquitto 2.0 (anonymous publish), MongoDB 7.0 (unauthenticated `listDatabases`) and MariaDB 11.4 (passwordless remote root) were then added without code (Table VI). Cores never exceeded two directives; 29% (MongoDB) to 62% (MariaDB) of pivots lay outside the core; P conflicted with 4–9 sources per service and left 17.5–22.5% of MongoDB and MariaDB verdicts wrong at full synchronization. The expert guard reproduced every held-out verdict and cut re-emulation from 93–100 to 28–30 per 100 edits. Learned guards matched it for MongoDB and MariaDB but over-fitted on Mosquitto, where operational keys entered the learned set (82% export), supporting the expert list as a floor. The specs exposed further semantics: on MariaDB 11.4 `skip-grant-tables` does not imply `skip-networking`, so a remote client logs in as root without a password, and Mosquitto 2.0 denies anonymous clients once a listener is declared unless `allow_anonymous` is set. Writing them also surfaced two harness pitfalls: YAML 1.1 parsers read `off` as a Boolean where mongod reads a string, and quiet log settings can hide a server's readiness line.

On public files for these services (414 Mosquitto, 168 MongoDB and 373 MariaDB distinct files), admission again exposed environment and version effects: 80 of 168 MongoDB files failed, 25 of them because removing the environment-bound `logAppend` invalidates a retained `logRotate: reopen` (a normalization artifact), others because of removed options such as `net.http`; 160 of 373 MariaDB files failed, mostly on paths and memory sizes the twin cannot provide. Of 368 admitted Mosquitto files, 182 (49%) accepted anonymous publishers in the twin setting. Cores stayed within two directives. Synthetic guards transferred without loss to MongoDB and MariaDB (60 sampled verdicts each, exporting 63% and 19%) but not fully to Mosquitto

(expert 95%, learned 92%), where misses involved keys outside the catalog, such as a listener switched to the WebSocket `protocol`. MariaDB also showed *validity pivots*: in 14 files, removing `character-set-server` left a mismatched `collation-server` and the server refused to start, so a line can protect a verdict simply by keeping the file valid.

The machinery is packaged as a command-line tool: `analyze` reports each verdict with its core, pivots labeled as enabling or protective, ineffective security lines and keys left to defaults (with SARIF output for code scanning); `export` emits the guard-key subset with *unset* markers; `fingerprint` compares builds' default behavior, flagging 8 of 19 Redis probe configurations that differ between the Ubuntu package and the official 7.0.15 image; and `gate` is the key-guarded change check evaluated in **RQ4**.

VII. Discussion and Limitations

Design rules. Five rules follow for twin platforms. (1) *Synchronize by guard keys*: export only directives whose keys are in the union of learned and expert guards, a quarter of a Redis file and an eighth of an sshd file in our public corpus, at no measured accuracy cost. (2) *Report absence*: send an explicit unset marker for every guard key, so platform defaults never stand in for vendor defaults (Proposition 1); this was the largest single error source in public data. (3) *Pin the build, not just the version*, and verify compiled defaults by querying the running twin. (4) *Guard re-emulation by key, with the expert list as a floor*: guards cut re-emulation three- to five-fold on synthetic drift and by 44% against directive hashing on real commits, with no stale expert verdicts, but a learned guard cannot know keys that never mattered in its sample. (5) *Attach cores and pivots to findings*: a core is a minimal explanation, the pivots a remediation map.

When must guards be relearned? A learned guard is valid only while locality holds for the configurations it meets. It must be relearned when the server build changes semantics (Redis 6.2 to 7.0 changed what `bind` means), when the probe or property changes, when the base profile changes, or when drift introduces keys never decisive in training; operationally, any audit that finds a stale verdict or an export that hits keys outside the training catalog should trigger relearning, which costs a few server runs per configuration (Algorithm 1).

Agreement is not correctness. All accuracies here are *empirical verdict agreement* with a real server on a finite corpus and one probe per property. Proposition 1 gives correctness only where locality holds, and locality is itself an empirical claim; agreement says nothing about attack paths the probe does not exercise, other builds or other client positions.

Safe remediation. Pivots are candidate fixes, not fixes. Removing an enabling pivot removed the exposure in the twin, but a line can also be a validity pivot, and a fix may break legitimate use; a remediation should be applied to a copy, re-verified for both the security property and a legitimate workflow (as our earlier platform's patch verifier does [1]), and only then proposed.

Demonstrated privacy benefit. On the public corpus, full synchronization would ship 10,707 Redis and 4,662 sshd directives; guard-key export ships 750 (7.0%) and 473 (10.1%), drawn from 4 of 149 and 5 of 110 distinct keys. Of 276 Redis and 460 sshd values that contain file paths, host names or addresses, only 1 and 5 are exported. pg_hba gains nothing, since every rule is security-relevant.

When is static analysis enough? For mature services a careful analyst can encode nearly everything; the residual errors cluster where documentation is easy to misread and where real files use unmodeled syntax. The two are complements: the analyzer is instant and explainable, the twin keeps it honest, and fidelity cores tell the twin what it may leave out. Above all, tiny cores sit alongside a slowly falling fidelity curve, because what a verdict needs lives in its pivots, not its core: fidelity belongs to a verdict, not to a file.

Threats to validity and ethics. Synthetic configurations follow postures we designed; public files over-represent containers, tutorials and templates, so their rates are not population estimates and core sizes describe our corpora, not a law. Normalization makes public files runnable but removes effects such as TLS-only listeners, and in a few cases (MongoDB `logAppend`) causes rejections itself. Each property is one wire question; richer ones (proxy chains, role-dependent privileges) would likely give larger cores. Twins run on one host, cannot model PAM, and leave firewall and multi-service effects out of scope. Guards were learned from 30 configurations per family, and the analyzers were written by the authors, so we report their errors as semantic corner cases rather than tool rankings. All servers ran in an isolated container with synthetic accounts; public files were read anonymously, secrets were replaced before storing anything, and released data identify configurations only by content hash, without repository names.

VIII. Conclusion

To *reproduce* a security verdict, a twin needs very little: at most two directives in every case we measured, across seven services, synthetic or public. To *keep* reproducing it as files change and platforms fill in defaults, a twin must track the larger set of directives whose removal would invert it, mark unset keys explicitly, and run the same build as the source, not just the same version. Treating fidelity as a property of verdicts rather than files lets a platform export a quarter of a configuration and re-emulate a fraction as often while agreeing with real servers on every public verdict we tested for the original families, and it explains why twins built from defaults, or from most of a file, can be confidently wrong. The harness, service specifications, corpora, raw results and analysis scripts accompany this paper.

References

[1] "Digital Twin: Discovery, emulation and bounded hunting," source code and evaluation artifacts, GitHub repository poojarishreyas/Digital_twin, 2026. [Online]. Available: https://github.com/poojarishreyas/Digital_twin

[2] M. Grieves and J. Vickers, "Digital twin: Mitigating unpredictable, undesirable emergent behavior in complex systems," in *Transdisciplinary Perspectives on Complex Systems*, F.-J. Kahlen, S. Flumerfelt, and A. Alves, Eds. Cham, Switzerland: Springer, 2017, pp. 85–113.

[3] F. Tao, H. Zhang, A. Liu, and A. Y. C. Nee, "Digital twin in industry: State-of-the-art," *IEEE Trans. Ind. Informat.*, vol. 15, no. 4, pp. 2405–2415, Apr. 2019.

[4] M. Eckhart and A. Ekelhart, "Towards security-aware virtual environments for digital twins," in *Proc. 4th ACM Workshop Cyber-Physical System Security (CPSS)*, 2018, pp. 61–72.

[5] M. Eckhart and A. Ekelhart, "Digital twins for cyber-physical systems security: State of the art and outlook," in *Security and Quality in Cyber-Physical Systems Engineering*, S. Biffl, M. Eckhart, A. Lüder, and E. Weippl, Eds. Cham, Switzerland: Springer, 2019, pp. 383–412.

[6] M. Dietz and G. Pernul, "Unleashing the digital twin's potential for ICS security," *IEEE Security & Privacy*, vol. 18, no. 4, pp. 20–27, 2020.

[7] D. Holmes, M. Papathanasaki, L. Maglaras, M. A. Ferrag, S. Nepal, and H. Janicke, "Digital twins and cyber security – solution or challenge?" in *Proc. 6th South-East Europe Design Automation, Computer Engineering, Computer Networks and Social Media Conf. (SEEDA-CECNSM)*, 2021, pp. 1–8.

[8] P. Empl, D. Koch, M. Dietz, and G. Pernul, "Digital twins in security operations: State of the art and future perspectives," *ACM Comput. Surv.*, vol. 58, no. 1, Art. no. 18, 2025.

[9] National Academies of Sciences, Engineering, and Medicine, *Foundational Research Gaps and Future Directions for Digital Twins*. Washington, DC, USA: The National Academies Press, 2024.

[10] H. H. Liu *et al.*, "CrystalNet: Faithfully emulating large production networks," in *Proc. 26th ACM Symp. Operating Systems Principles (SOSP)*, 2017, pp. 599–613.

[11] M. M. Yamin, B. Katt, and V. Gkioulos, "Cyber ranges and security testbeds: Scenarios, functions, tools and architecture," *Computers & Security*, vol. 88, Art. no. 101636, Jan. 2020.

[12] A. Fogel, S. Fung, L. Pedrosa, M. Walraed-Sullivan, R. Govindan, R. Mahajan, and T. Millstein, "A general approach to network configuration analysis," in *Proc. 12th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2015, pp. 469–483.

[13] R. Beckett, A. Gupta, R. Mahajan, and D. Walker, "A general approach to network configuration verification," in *Proc. ACM SIGCOMM*, 2017, pp. 155–168.

[14] R. Birkner, D. Drachsler-Cohen, L. Vanbever, and M. Vechev, "Config2Spec: Mining network specifications from network configurations," in *Proc. 17th USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2020, pp. 969–984.

[15] X. Ou, S. Govindavajhala, and A. W. Appel, "MulVAL: A logic-based network security analyzer," in *Proc. 14th USENIX Security Symp.*, 2005, pp. 113–128.

[16] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, "Automated generation and analysis of attack graphs," in *Proc. IEEE Symp. Security and Privacy*, 2002, pp. 273–284.

[17] Z. Yin, X. Ma, J. Zheng, Y. Zhou, L. N. Bairavasundaram, and S. Pasupathy, "An empirical study on configuration errors in commercial and open source systems," in *Proc. 23rd ACM Symp. Operating Systems Principles (SOSP)*, 2011, pp. 159–172.

[18] T. Xu, L. Jin, X. Fan, Y. Zhou, S. Pasupathy, and R. Talwadker, "Hey, you have given me too many knobs!: Understanding and dealing with over-designed configuration in system software," in *Proc. 10th Joint Meeting European Software Engineering Conf. and ACM SIGSOFT Symp. Foundations of Software Engineering (ESEC/FSE)*, 2015, pp. 307–319.

[19] H. J. Wang, J. C. Platt, Y. Chen, R. Zhang, and Y.-M. Wang, "Automatic misconfiguration troubleshooting with PeerPressure," in *Proc. 6th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2004, pp. 245–258.

[20] J. Zhang, L. Renganarayana, X. Zhang, N. Ge, V. Bala, T. Xu, and Y. Zhou, "EnCore: Exploiting system environment and correlation information for misconfiguration detection," in *Proc. 19th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2014, pp. 687–700.

[21] T. Xu, X. Jin, P. Huang, Y. Zhou, S. Lu, L. Jin, and S. Pasupathy, "Early detection of configuration errors to reduce failure damage," in *Proc. 12th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2016, pp. 619–634.

[22] L. Keller, P. Upadhyaya, and G. Candea, "ConfErr: A tool for assessing resilience to human configuration errors," in *Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, 2008, pp. 157–166.

[23] A. Rabkin and R. H. Katz, "Precomputing possible configuration error diagnoses," in *Proc. 26th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, 2011, pp. 193–202.

[24] X. Sun, R. Cheng, J. Chen, E. Ang, O. Legunsen, and T. Xu, "Testing configuration changes in context to prevent production failures," in *Proc. 14th USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 2020, pp. 735–751.

[25] S. Wang, X. Lian, D. Marinov, and T. Xu, "Test selection for unified regression testing," in *Proc. 45th IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2023.

[26] S. Wang, X. Lian, Q. Li, D. Marinov, and T. Xu, "Ctest4J: A practical configuration testing framework for Java," in *Companion Proc. 32nd ACM Int. Conf. Foundations of Software Engineering (FSE Companion)*, 2024, pp. 562–566.

[27] S. Böhm, F. Sattler, N. Siegmund, and S. Apel, "Detecting performance-relevant changes in configurable software systems," arXiv:2511.17271, 2025.

[28] G.-P. Drosos, T. Sotiropoulos, G. Alexopoulos, D. Mitropoulos, and Z. Su, "When your infrastructure is a buggy program: Understanding faults in infrastructure as code ecosystems," *Proc. ACM Program. Lang.*, vol. 8, no. OOPSLA2, Art. no. 359, 2024.

[29] N. Saavedra, J. F. Ferreira, and A. Mendes, "InfraFix: Technology-agnostic repair of infrastructure as code," in *Companion Proc. 34th ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA Companion)*, 2025.

[30] N. Siegmund, S. S. Kolesnikov, C. Kästner, S. Apel, D. Batory, M. Rosenmüller, and G. Saake, "Predicting performance via automated feature-interaction detection," in *Proc. 34th Int. Conf. Software Engineering (ICSE)*, 2012, pp. 167–177.

[31] M. Sayagh, N. Kerzazi, B. Adams, and F. Petrillo, "Software configuration engineering in practice: Interviews, survey, and systematic literature review," *IEEE Trans. Softw. Eng.*, vol. 46, no. 6, pp. 646–673, 2020.

[32] A. Rahman, C. Parnin, and L. Williams, "The seven sins: Security smells in infrastructure as code scripts," in *Proc. 41st IEEE/ACM Int. Conf. Software Engineering (ICSE)*, 2019, pp. 164–175.

[33] C. Tang *et al.*, "Holistic configuration management at Facebook," in *Proc. 25th ACM Symp. Operating Systems Principles (SOSP)*, 2015, pp. 328–343.

[34] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE Trans. Softw. Eng.*, vol. 28, no. 2, pp. 183–200, Feb. 2002.

[35] R. Hildebrandt and A. Zeller, "Simplifying failure-inducing input," in *Proc. ACM SIGSOFT Int. Symp. Software Testing and Analysis (ISSTA)*, 2000, pp. 135–145.

[36] G. Misherghi and Z. Su, "HDD: Hierarchical delta debugging," in *Proc. 28th Int. Conf. Software Engineering (ICSE)*, 2006, pp. 142–151.

[37] R. Hodován and Á. Kiss, "Practical improvements to the minimizing delta debugging algorithm," in *Proc. 11th Int. Joint Conf. Software Technologies (ICSOFT-EA)*, 2016, pp. 241–248.

[38] M. Weiser, "Program slicing," *IEEE Trans. Softw. Eng.*, vol. SE-10, no. 4, pp. 352–357, Jul. 1984.